Integration of General-Purpose Training Infrastructures (AWS, Google, Meta) with the Rapidus Inference Accelerator Using Open-Source Compiler Toolchains

Excerpt from the paper : Strategic Considerations of the Gate-All-Around (GAA) Transistor for AI Semiconductors in Software-Defined Vehicles (SDVs) and Robotic Manipulation Systems: latest revised on Dec. 04, 2025

Date: July 24, 2026

Section No.1 : Open-Source Compiler Ecosystem for Rapidus GAA

To advance the establishment of an open-source compiler ecosystem for Rapidus GAA, three representative open-source compilers and AI frameworks are introduced.

＊OpenCL: A low-level, general-purpose foundation for parallel computing. (It enables computation across diverse devices such as CPUs, GPUs, and FPGAs.)
＊SYCL: A high-level C++ API that builds on OpenCL, offering improved usability. (It is vendor-independent and supports a wide range of hardware platforms including Intel, AMD, and NVIDIA.) (see Section No.2)
＊TVM: A compiler stack designed to optimize and deploy deep learning models across various hardware targets. (It features a user-friendly Python-based API.)

SYCL, in particular, shows the highest affinity with the philosophy of RISC-V, which is not constrained by a specific vendor's ecosystem, in terms of its vendor-independent nature and high-level C++ descriptiveness. For Rapidus to realize the superiority of GAA in the market, it is considered indispensable to promote an 'open-source compiler strategy' centered on SYCL.

Section No.2 : ONNX Execution Provider as the Core Technology for Physical AI. Implementing Inference on SoCs Incorporating Semiconductors such as Rapidus/Tenstorrent and Training in the Cloud (AWS, Google, Meta)

Google TPU and Amazon Trainium, which are general-purpose training accelerators, are the primary candidates for the large-scale AI model training infrastructure. These are primarily intended for large-scale AI training, similar in positioning to NVIDIA GPUs (A100/H100).

When examining the specific requirements for connecting Training and Inference, various technical requirements exist. Google TPUs surpass Amazon Trainium in terms of performance and maturity; however, TPUs architecture’s reliance on XLA (Accelerated Linear Algebra) introduces challenges for ONNX (Open Neural Network Exchange) compatibility. On the other hand, since Trainium utilizes the Neuron SDK as its optimization pipeline, it allows for the conversion of models into the ONNX format. (Amazon’s AI chips: Trainium for training, Inferentia for inference.)

Accordingly, the subsequent analysis illustrates a case predicated on the adoption of Trainium as the foundational training platform; however, it is important to note that Trainium does not directly support ONNX, and its integration with ONNX Runtime is still in the developmental phase.

(Currently, the use of ONNX models may require converting them back to frameworks such as PyTorch before processing with Neuron, so it cannot be regarded as fully 'ONNX-native' support.)

It is paramount that the multimodal models created on the training side (cloud) can be executed accurately and efficiently on the SoC incorporating semiconductors such as Rapidus/Tenstorrent on the inference side (edge). Ref.: AWS Trainium (Trn2,3) enables training at 30–50% lower cost compared to GPUs.

****

Goal: To enable the inference SoC to directly execute ONNX models trained on Amazon Trainium or similar platforms, without the need for cumbersome instruction code rewriting.
Method of realization: Utilize the ONNX Execution Provider (EP) to seamlessly link the operators of the trained model with the hardware of the inference SoC.

① Efficiency Improvement of Rapidus/Tenstorrent SoC Execution (Role of ONNX EP)
To achieve high performance and low power consumption with the new SoC architecture of Rapidus and Tenstorrent, optimization of the core Transformer operations by the ONNX EP is essential.

＊Mechanism of ONNX EP:
・The EP can be implemented in C++, allowing each company to add custom EPs tailored to their own semiconductors.
・The EP parses the operators of the trained model and maps them to kernels and libraries optimized for the inference SoC hardware (Rapidus/Tenstorrent) for execution.

＊Division of Roles within the SoC: (e.g. Graph Partition Strategy)
" Role of Rapidus " (Dedicated Accelerator):
・Prioritizes the execution of high-load core operations such as matrix operations, Attention mechanism, normalization, and quantization.
・GEMM/MatMul and Attention-related operations are preferentially mapped to Rapidus as the core for acceleration.
" Role of Tenstorrent " (RISC-V CPU):
・Handles supplementary processing such as control systems, shaping / reshaping, and fallback for unsupported operations by Rapidus.

＊Core Operator Set (Primary inference Path for Transformer Inference):
・GEMM/MatMul, Attention-related operations, Normalization, Activation, Quantize/Dequantize (Q/DQ), and Elementwise basic set are preferentially mapped to Rapidus.

② Unified Policy for Numerical Format and Precision
Strict numerical processing rules are established to balance model performance and accuracy.

＊Priority Precision:
・Rapidus primarily utilizes high-efficiency FP8 and BF16.
・Tenstorrent CPU uses BF16 as a base and switches to FP32 if necessary.

・INT8 is selectively adopted for specific processes within the Attention mechanism, such as QK (Query-Key) and Value projection.

＊Integration of Quantization Processing:

・Quantization processes like FP8/INT8 are adjusted to the precision characteristics of the Rapidus AI accelerator within the ONNX EP internal processing.
・Per-channel scaling is prioritized, with rescaling performed at the LayerNorm boundary.
・Quantization/Dequantization (Q/DQ) metadata is stored externally to the ONNX format, and the EP manages and references it in an integrated manner.

＊Softmax Approximation Rule:

・The balance between speed and precision is guaranteed by pre-defining an approximation tolerance (e.g. maximum relative error of 0.001) and gatekeeping decoding quality through CI (Continuous Integration).

③ Real-Time Processing and Latency (Low Delay)

Inference at the edge often requires extremely rapid responses to sensor input.

＊Guarantee of Low Latency:

・The mapping of operators by the ONNX EP must be designed to prioritize minimizing the delay (latency) from the start to the completion of inference, not just maximizing throughput (processing capacity).
・For sequential operations like Attention (e.g., in the decoder part), preemptive task scheduling coordinated with a Real-Time OS (RTOS) becomes important.

④ Maturity of Development Efficiency (MLOps)

The maturity of the SDK (including drivers, memory management, and performance measurement tools) that constitutes the ONNX EP is directly linked to the operational efficiency and reliability for developers integrating it into SDVs (Software-Defined Vehicles) and robotics.

⑤ Security and Functional Safety (Safety & Security)

In embedded systems for automobiles (SDVs) and physical AI, not only performance and power consumption but also system reliability and safety design are essential requirements.

＊Functional Safety (ISO 26262 / IEC 61508):

・The Rapidus/Tenstorrent SoC architecture and ONNX EP must implement redundancy mechanisms and error detection/correction functions to ensure the safety of inference results.
・Custom kernels and libraries invoked by the EP must undergo a certification process that meets safety standards such as ASIL (Automotive Safety Integrity Level).

＊Security (Cyber Security):

・Hardware security features such as Secure Boot and encryption must be integrated into the EP's load path to prevent tampering with inference models and quantization metadata, thereby ensuring the protection of intellectual property and the system.

****

With regard to the current Meta MTIA, it is not a general-purpose training accelerator like Trainium, but a dedicated ASIC optimized for Meta's specific workloads, while Trainium is a chip primarily intended for large-scale AI model training; therefore, it is inferred that cloud training will be limited to Google and Amazon.

****

ONNX EP-based operator mapping plays an important role in connecting the inference model to the execution environment of the target SoC. Meanwhile, the optimization required to fully exploit the performance of the actual hardware is not completed by ONNX EP alone; rather, it is determined, from the lowering of ONNX to TOSA onward, through HW/SW co-design between the Compiler/Backend/Runtime and the NN accelerator hardware. In other words, building an integrated execution path from ONNX EP as the entry point through the hardware-specific optimization below TOSA is the key to efficient inference execution.(See Appendix for details)

In conclusion, SYCL abstracts program portability for developers (see Section No.1), while ONNX EP abstracts inference portability across environments. The ONNX Execution Provider functions not merely as a compatibility layer, but as a logical bridge connecting the cloud and the edge (physical AI, autonomous driving system and SDV)

Section No.3 : Implementing Inference Model Deployment on the Rapidus/Tenstorrent SoC

The method of transferring a model trained on the NVIDIA cloud (GPU/CUDA) environment to the the Rapidus/Tenstorrent inference SoC exhibits distinct differences in terms of model compatibility, optimization difficulty, and performance retention compared to the path utilizing AWS Trainium via ONNX.

In the NVIDIA environment, proprietary NVIDIA libraries and kernels, such as CUDA, cuDNN, and TensorRT during inference, are heavily employed during the post-training optimization phase to achieve high model performance. Consequently, when exporting a model optimized within the NVIDIA ecosystem to the ONNX format, there is a risk of including NVIDIA-specific extended operators (custom operations).

If such proprietary operators are included, the Execution Provider (EP) on the Rapidus/Tenstorrent inference SoC side may fail to recognize them, potentially leading to a fallback to the Tenstorrent RISC-V CPU or, in the worst case, rendering execution impossible. Conversely, the path utilizing platforms like AWS Trainium or Google TPU (as discussed in Section No.2) is predicated on the use of ONNX as a common intermediate language. Furthermore, AWS Trainium provides a model optimization pipeline via the Neuron SDK and supports conversion to the ONNX format. By leveraging this path, the Rapidus/Tenstorrent inference SoC development team can focus intensely on the development of the ONNX EP.

Following training on Trainium, the conversion of the model into the open, standard format of ONNX

allows the Rapidus/Tenstorrent inference SoC's ONNX Execution Provider (EP) to process the model graph while expecting a set of pure ONNX operators. As a result, the respective roles of training (cloud) and inference (edge) are clearly separated by the ONNX standard.

The role of the EP is critical. For instance, upon encountering an ONNX operator such as " Matrix Multiplication (MatMul) ", the EP must determine and execute the precise combination of custom instructions, memory management methods, and FP8/BF16 precision on the AI accelerator built using Rapidus's 2nm GAA process that will yield the fastest and most power-efficient execution.

Only Rapidus and Tenstorrent, which are responsible for the design and manufacturing, possess the necessary knowledge of the custom hardware's internal architecture and its optimized kernel libraries. Rapidus/Tenstorrent does not need to develop the entire ONNX Runtime (ORT). However, since ORT itself is open-source, the development of the following custom modules is required.

Module-I) Rapidus-Specific ONNX EP:
The C++ component responsible for parsing ONNX operators and performing graph partitioning for the Rapidus accelerator (including delegation of processing to Tenstorrent components).

Module-II) Custom Kernel Library for the Rapidus Accelerator:
The low-level implementation code for high-speed execution of fundamental operations like GEMM (General Matrix Multiply) and Attention on the Rapidus accelerator including optimizations for FP8/BF16 support. (see Section No.2)

****

From both a technical and strategic perspective, the path utilizing AWS Trainium, which leverages ONNX as the common, open intermediate format, is determined to be more aligned with the the Rapidus/Tenstorrent inference SoC's design philosophy and significantly easier to implement.

I affirm that I have no affiliations—direct or indirect—with any automotive corporation, semiconductor enterprise, political entity, or religious organization. For reasons of confidentiality, I respectfully request that this material be kindly destroyed after it has been read.

With kindest regards,
Akira. Ver.4-3 Revised July 30, 2026